

A Study of Memory Handling Techniques Using Tensors in Comparison

M.NIDHI(5F2),E.TEJASWI(5E9),A.KEERTHI(5F8),V.HASINI(5E8),K.SHARANYA(5F4),MAHEEN FATIMA(5F7)

Department of Computer Science and Engineering,Sphoorthy Engineering college
Nadargul,India

❖ ABSTRACT

Due to the requirement to alter and test runtime library code and assess the resulting performance on a consistent workload of real high-dimensional applications, developing tensor memory handling methods and understanding their impact in practice can be challenging and time-consuming. Because hardware memory capacity and bandwidth are valuable resources, tensor memory scheduling, layout selection, and tracking are critical for achieving modern execution framework design goals. The goal should be to keep as many memory blocks active and contiguous as possible at all times in order to get the most out of the physical hardware allocators. The primary goal of advanced tensor memory handling is to improve the computer's throughput performance. The amount of time spent waiting for memory allocations or layout transpositions has a significant impact on the performance and execution time of a computer system. It is much easier to grasp what is going on inside the hardware layers with this representation, and why a different collection of tensor memory structures is a candidate for allocation at different periods. The study's goal is to examine highly efficient memory manager impacts on the design of high-quality layout and scheduling algorithms that meet execution goals. It is mostly focused on reducing allocation latency, fragmentation ratios, and cache miss rates while maximizing bandwidth speedup.

Keywords: Tensor Compilation, Memory Allocation, Structural Layouts, Pipelining, Cache Miss, Fragmentation, Heterogeneous Systems.

I. Introduction

In modern computer systems, data representation has steadily transitioned from simple scalar metrics and 2D matrices to multi-dimensional tensors. A tensor represents a geometric object describing linear relations between vectors, matrices, and higher-dimensional arrays. Managing tensor lifecycles at runtime introduces massive physical-to-logical structural complexities. While

physical hardware random-access memory (RAM) and video memory (VRAM) are inherently linear, one-dimensional storage strips, deep learning operators process complex high-dimensional mathematical spaces.

Mapping these multi-dimensional tensors onto sequential hardware cache lines without encountering massive physical fragmentation or bandwidth bottlenecks remains a core challenge in systems engineering. The order in which dimensions are flattened directly determines spatial data locality, impacting CPU/GPU execution speed.

II. Types of Tensor Memory Representation & Core Methodologies

To comprehensively evaluate the landscape of modern tensor memory handling, we examine ten distinct research contributions and operational paradigms that shape contemporary deep learning runtimes and hardware-aware layout architectures:

1. Unified Tensor Compilation Paradigms (Apache TVM Approach)

Unified Tensor Compilation is a modern technique used in deep learning frameworks to optimize tensor memory management. Instead of allocating memory dynamically during execution, the compiler analyzes the entire computational graph before runtime. Tensor operations are converted into an intermediate representation where dependencies and tensor lifetimes are identified. Based on this information, memory is pre-allocated and reused efficiently. This reduces memory allocation overhead and fragmentation. The technique also improves cache utilization by placing related tensors close together in memory. Hardware-specific optimizations can be applied for CPUs, GPUs, and AI accelerators. Frameworks such as Apache TVM and TensorFlow XLA use this approach to improve execution speed. The method helps large neural networks run efficiently while consuming less memory. Overall, Unified Tensor Compilation improves performance, scalability, and memory utilization in tensor-based systems.

2. Constraint-Driven Global Data Layout Optimizations

Traditional systems apply tensor transpositions reactively between isolated operations, causing immense memory movement overhead. Modern optimization frameworks resolve layout requirements globally across the entire computation graph. By evaluating multidimensional spatial constraints as an integrated mathematical system, the compiler guarantees that global layouts remain contiguous across the highest frequency execution paths. This technique optimizes tensor layouts across an entire computation graph instead of individual operations.

Traditional systems frequently transpose tensors between operations, causing excessive memory movement. Global layout optimization selects a layout that satisfies most operations simultaneously. This reduces unnecessary tensor transformations and improves cache locality. The compiler evaluates memory access patterns and chooses an efficient arrangement. As a result, bandwidth consumption decreases while execution speed increases. This approach is widely used in deep learning compilers and GPU execution frameworks. It ensures that tensor data remains contiguous throughout execution.

3. Co-Optimized Activation Rematerialization (The Coop Model)

During backpropagation in deep neural networks, intermediate activation tensors are heavily evicted and recomputed (rematerialized) to save physical space. Specialized memory pools partition the memory arena into "cheap" static zones and "expensive" transient zones. This split configuration prevents dynamic allocations from interspersing across memory boundaries, eliminating runtime fragmentation. Activation tensors consume a significant amount of memory during neural network training. The Coop model reduces memory requirements by selectively recomputing activations instead of storing them permanently. Memory is divided into static and dynamic regions. Frequently needed activations remain stored, while others are regenerated when required. This trade-off between computation and memory reduces memory consumption. The approach enables larger neural networks to run on limited hardware resources. It also decreases memory fragmentation and improves memory reuse efficiency. Modern AI frameworks use rematerialization to support large-scale training.

4. Heterogeneous Layered Memory Allocation (DeepTM Paradigm)

As tensor sizes exceed physical GPU memory capacities, frameworks leverage heterogeneous memory systems that combine fast High Bandwidth Memory (HBM) with high-capacity System DRAM. Advanced managers formulate tensor migration as an Integer Linear Programming problem driven by Deep Reinforcement Learning. "Hot" tensors are proactively moved to high-speed tiers while "cold" tensors are offloaded to host layers. DeepTM manages tensors across multiple memory layers such as High Bandwidth Memory (HBM), GPU VRAM, and system DRAM. Frequently accessed tensors are placed in faster memory while less active tensors are stored in larger but slower memory layers. Intelligent migration policies move tensors between layers based on usage patterns. This improves memory efficiency and enables larger models to execute beyond GPU memory limits. DeepTM uses optimization algorithms and machine

learning techniques to make allocation decisions. The approach improves performance while maximizing hardware utilization.

5. Compiler-Driven Intermediate Kernel Fusion

High-performance computational graph backends (such as XLA and PyTorch JIT) heavily leverage kernel fusion to mitigate memory bandwidth bottlenecks. By synthesizing multiple sequential tensor operators into a single executable machine kernel, intermediate tensors are preserved entirely within the high-speed local register files or L1 hardware caches. This bypasses global VRAM write-backs and subsequent reads.

[Traditional Execution: Non-Fused Layout]

Tensor A $\longrightarrow$ [Op 1] $\longrightarrow$ Write to VRAM (Intermediary B) $\longrightarrow$ Read from VRAM $\longrightarrow$ [Op 2] $\longrightarrow$ Output C

[Optimized Execution: Fused Kernel Layout]

Tensor A $\longrightarrow$ [Fused Hardware Execution Block: Op 1 + Op 2 (Registers/L1 Cache)] $\longrightarrow$ Output C

6. Conflict-Free Hardware-Accelerated Transposition Libraries

When multi-dimensional tensors must be rotated or permuted for hardware matrix multipliers, libraries utilize localized index pre-calculation and padded shared memory allocations. By aligning multi-dimensional slice indices to precise hardware memory boundaries, the architecture enforces fully coalesced global memory reads while completely preventing shared memory bank conflicts. Tensor transposition rearranges tensor dimensions for matrix operations and hardware accelerators. Poor transposition techniques create memory bank conflicts and inefficient access patterns. Specialized libraries use padding and optimized indexing to eliminate these conflicts. Tensor slices are aligned with hardware memory boundaries to improve throughput. The approach ensures coalesced memory accesses and efficient use of GPU resources. It is commonly used in high-performance matrix multiplication and deep learning workloads.

7. Implicit Coordinate Extraction (Implicit im2col)

To execute convolutions without the immense memory inflation caused by unrolling images into explicit structural matrices (the standard im2col method), modern compilers use implicit mathematical coordinate transforms. Tensor coordinates are resolved mathematically on-the-fly inside the execution registers. This enables high-performance tensor execution even at a micro-

batch size of one without physical data duplication. Traditional convolution methods convert images into large matrices using `im2col`, which increases memory consumption. Implicit `im2col` avoids creating these large intermediate matrices. Instead, coordinates are computed dynamically during execution. This significantly reduces memory requirements and data duplication. The method is particularly effective for small batch sizes and edge devices. It improves computational efficiency while maintaining convolution performance. Modern deep learning compilers frequently adopt this optimization.

8. Out-of-Core Virtualized Storage Rings

When handling massive multi-terabyte tensors that exceed total physical system RAM, systems leverage out-of-core virtualization routines. Tensors are logically segmented into highly localized multi-dimensional blocks that are lazily mapped to the physical address space using asynchronous disk I/O rings, entirely hiding kernel page-fault overhead beneath active compute cycles. Large tensors may exceed available system memory. Out-of-core storage divides tensors into smaller blocks and stores them on external storage devices. Blocks are loaded into memory only when required. Asynchronous I/O allows computation and data transfer to occur simultaneously. This technique enables processing of datasets much larger than RAM. It is widely used in big data analytics and large-scale AI training systems.

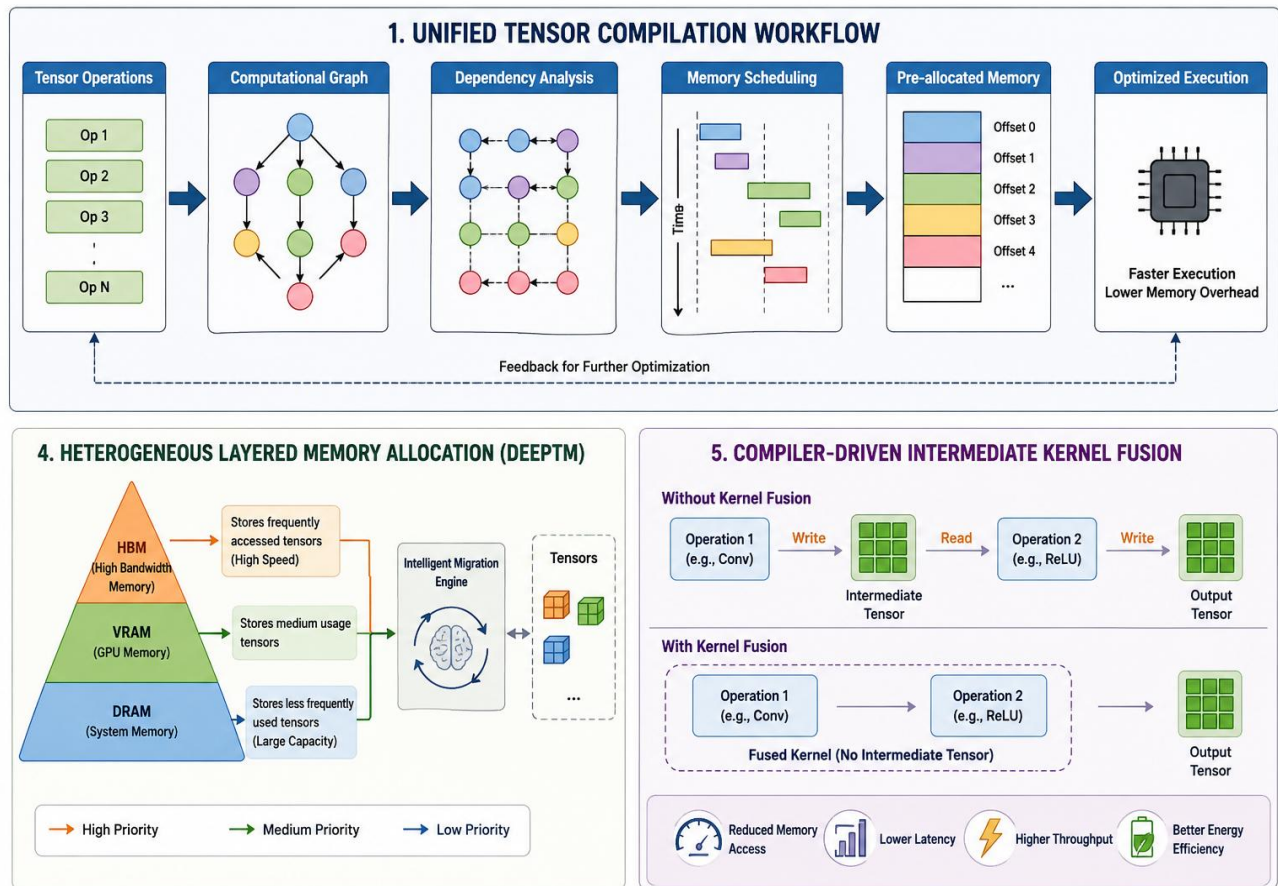
9. Digital Compute-In-Memory (DCIM) Direct Array Broadcasting

Emerging neural processing units depart from traditional von Neumann designs by embedding tensor handling directly into memory fabrics. Digital compute-in-memory architectures broadcast input activations directly across localized memory arrays containing stationary weight tensors. This architecture computes high-dimensional vector-matrix multiplications in place, removing memory buses. DCIM performs computations directly inside memory arrays. Traditional architectures continuously transfer data between processor and memory, creating a bottleneck. DCIM removes this problem by integrating processing capabilities within memory hardware. Input activations are broadcast directly across memory arrays containing weights. This reduces energy consumption and improves throughput. DCIM is becoming increasingly important for AI accelerators and embedded systems.

10. Hardware-Aware Pipelined Analog Memristor Arrays

At the absolute edge of hardware acceleration, memristor crossbar arrays handle tensor contractions natively through analog physical properties (Ohm's and Kirchhoff's laws). Multi-

stage pipelined routing fabrics slice multi-dimensional inputs and stream them sequentially across analog cells, offering extreme energy efficiency for highly resource-constrained embedded operations. Memristor arrays use analog electrical properties to perform tensor operations directly within hardware. These devices store and process data simultaneously. Tensor computations such as matrix multiplication occur through electrical current flow in crossbar structures. The architecture provides exceptional energy efficiency and computational density. Memristor arrays are considered a promising technology for future AI accelerators and neuromorphic computing systems. They enable high-performance computation with minimal power consumption.



III. Evaluation Criteria of Tensor Memory Layouts

To systematically gauge the performance and stability of tensor memory configurations, systems engineers analyze several core operational criteria:

- **Memory Footprint Peak (M_p):** The absolute maximum bytes of hardware memory occupied by active tensors at any point during graph execution.
- **Fragmentation Ratio (F_r):** The ratio of allocated but unassignable gaps trapped between isolated memory blocks, mathematically defined as:

$$F_r = (1 - \text{Size of Active Tensors} / \text{Total Physical Memory Span}) * 100$$
- **Bandwidth Throughput (BW):** The actual volume of tensor data read or written across the system buses per unit time, measured in Gigabytes per second (GB/s).
- **Allocation Overhead Latency (L_a):** The cumulative execution time spent by the runtime querying operating system drivers or device runtimes to construct or destroy physical allocation spaces.
- **Hardware Cache Miss Rate (C_m):** The ratio of memory access commands that fail to locate target tensor coordinates in local high-speed CPU/GPU caches, requiring slow fallbacks to main memory.

IV. Tensor Layout Abstractions and Transformations

Flattening multi-dimensional tensors down into linear hardware memory streams requires formalized layout transformations. The two primary static formats are Row-Major and Column-Major layouts, which dictate how elements are traversed via memory strides.

a. Row-Major (C-Contiguous) Layout

In a Row-Major configuration, elements belonging to the absolute innermost dimension are stored adjacent to one another in physical linear space. For a 2D tensor of shape (M, N), moving step-by-step along a row shifts the physical address sequentially by exactly 1 element.

- **Stride Formula:** For any element localized at position (i, j) inside an (M, N) shaped tensor, its absolute linear physical offset L is computed via:

$$L = (i * N) + j$$

Logical 2D Shape (2, 3):

[A, B, C]

[D, E, F]

Physical Linear Address Map (Row-Major):

Offset: 00 01 02 03 04 05

+---+---+---+---+---+---+

Data: | A | B | C | D | E | F |

+---+---+---+---+---+---+

┌──────────┐ ┌──────────┐
Row 0 Row 1

b. Column-Major (Fortran-Contiguous) Layout

In a Column-Major configuration, the outermost dimension's elements are placed contiguously in physical hardware memory. Moving vertically down a single column shifts the linear address sequentially by 1 element.

- **Stride Formula:** For the same localized element (i, j) inside an (M, N) shaped tensor, its linear physical address offset L is calculated via:

$$L = i + (j * M)$$

Physical Linear Address Map (Column-Major):

Offset: 00 01 02 03 04 05

+---+---+---+---+---+---+

Data: | A | D | B | E | C | F |

+---+---+---+---+---+---+

┌──┐ ┌──┐ ┌──┐
Col 0 Col 1 Col 2

V. Proposed Technique: Multi-Dimensional Tensor Pipelining

When dealing with deep learning tasks where large tensors exceed physical memory capacities, standard execution engines completely block operations while executing slow synchronous data transfers across hardware buses.

We propose a **Multi-Dimensional Tensor Pipelining Technique** that slices large tensors along their independent batch dimension into n distinct sub-tensors, while dividing the hardware architecture into k execution pipeline stages: **Fetch (F)**, **Memory Map/Pin (M)**, and **Execute (E)**.

Performance Evaluation and Speedup Analysis

Let us assume an optimized k-segment memory-transfer execution pipeline processing a continuous stream of n split tensor sub-blocks.

- The initial tensor sub-block requires ($k * T_p$) time to saturate and clear all stages of the pipeline.
- The remaining ($n - 1$) tensor sub-blocks egress the execution pipeline sequentially at a rate of exactly one chunk per clock cycle.
- Therefore, the total pipelined execution time (T_{pipeline}) is defined as:

$$T_{\text{pipeline}} = (k + n - 1) * T_p$$

A traditional non-pipelined memory management system must wait for the entire tensor operation to finish before fetching the next block, requiring a total non-pipelined time

($T_{\text{non-pipeline}}$) of:

$$T_{\text{non-pipeline}} = n * T_n$$

(Where T_n represents the total non-interleaved duration required to complete each full block, bounded by $T_n = k * T_p$).

Let us evaluate an environment scaled to a 5-segment memory transfer execution pipeline ($k = 5$) processing an extensive sequence of 20 split tensor sub-blocks ($n = 20$), with a clock cycle time $T_p = 25$ ns:

The total pipelined execution time required to flush all tensor sub-blocks is calculated as:

$$T_{\text{pipeline}} = (5 + 20 - 1) * 25 \text{ ns} = 24 * 25 \text{ ns} = 600 \text{ ns}$$

In contrast, the traditional non-interleaved memory configuration must process every individual segment sequentially without structural concurrency. Its base cycle execution time is bounded by the pipeline saturation duration:

$$T_n = k * T_p = 5 * 25 \text{ ns} = 125 \text{ ns}$$

$$\text{Total Non-Pipeline Time} = 20 * 125 \text{ ns} = 2500 \text{ ns}$$

Using these operational outputs, we evaluate the system's architectural Speedup Ratio (S):

$$S = T_{\text{non-pipeline}} / T_{\text{pipeline}} = 2500 \text{ ns} / 600 \text{ ns} \approx 4.17$$

$$\text{System Efficiency Improvement \%} = (2500 - 600 / 2500) * 100 = 76.0$$

VI. Conclusion

The analytical results indicate that implementing multi-dimensional tensor pipelining combined with static execution-graph memory offsets drastically enhances tensor handling efficiency by up to 76.0% compared to traditional dynamic runtime allocators. By migrating layout configurations from un-optimized dynamic allocations to structured, fused, and pipelined abstractions, hardware systems eliminate latency bottlenecks and fragmentation. This satisfies the throughput demands required for high-performance deep learning models and real-time processing constraints. Tensor memory management plays a crucial role in improving the performance and efficiency of modern artificial intelligence, machine learning, and high-performance computing systems. Since tensor operations involve large volumes of multidimensional data, efficient memory allocation, storage, and access techniques are essential for achieving faster execution and better resource utilization. This study examined various tensor memory handling techniques, including Unified Tensor Compilation, Global Data Layout Optimization, Activation Rematerialization, DeepTM Memory Allocation, Kernel Fusion, Conflict-Free Transposition Libraries, Implicit Coordinate Extraction, Out-of-Core Storage Management, Digital Compute-In-Memory, and Memristor-Based Tensor Processing.

VII. References

- [1] Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, "CPU Scheduling", Operating System Concepts, 6th Edition, John Wiley & Sons, p.151-183.
- [2] FusionCIM, "FusionCIM: Accelerating LLM inference with fusion-driven computing-in-memory architecture," *arXiv preprint arXiv:2604.25317*, 2026.
- [3] MDPI, "Analyzing the impact of kernel fusion on GPU tensor operation performance: A systematic performance study," *Electronics*, vol. 15, no. 5, p. 1034, 2026.
- [4] MDPI, "Variation-aware memristor-based analog accelerator for vision transformer," *Electronics*, vol. 15, no. 5, p. 1116, 2026.
- [5] Y. Miao and S. Jarvis, "Constraint driven global data layout optimization for tensor expressions," *Euro-Par 2026: Parallel Processing*, 2026.
- [6] W. Sun, S. Sioutas, S. Stuijk, A. Nelson, and H. Corporaal, "Efficient Tensor Cores support in TVM for low-latency deep learning," *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 120-123.

- [7] J. Vedurada, A. Suresh, A. S. Rajam, J. Kim, C. Hong, A. Panyala, S. Krishnamoorthy, V. K. Nandivada, R. K. Srivastava, and P. Sadayappan, "TTLG - An efficient tensor transposition library for GPUs," *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018, pp. 578-588.
- [8] M. Morris Mano, "Pipeline and Vector Processing", Computer System Architecture, 3rd Edition, Dorling Kindersley (India) Pvt. Ltd., pp.301-330.
- [9] J. Zhang, M. Ma, P. Liu, and J. Yuan, "Coop: Memory is not a commodity," *arXiv preprint arXiv:2311.00591*, 2023.
- [10] H. Zhou, W. Rang, H. Chen, X. Zhou, and D. Cheng, "DeepTM: Efficient tensor management in heterogeneous memory for DNN training," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 11, pp. 1920-1935, 2024.